Emotion Classification in Short Texts using Fine-Tuned Transformer Models

Ofek Raban, Ron Gabay
Bar-Ilan University
Our code is available here.

1 Introduction

Emotion classification from text focuses on identifying the emotional state expressed in a written sentence. In this project, we study short and informal social media texts, specifically Twitter posts, and classify each input into one of six emotion categories: sadness, joy, love, anger, fear, and surprise. This task is challenging due to the limited context of short texts, noisy and informal language, and a clear class imbalance, where some emotions appear much less frequently than others.

In recent years, pretrained Transformer models have become the dominant approach for many Natural Language Processing tasks (Vaswani et al., 2017). These models learn rich contextual representations from large corpora and can be adapted to downstream tasks through fine tuning (Devlin et al., 2019). Compared to recurrent architectures such as LSTM and GRU, Transformer based models generally achieve better performance on short text classification, mainly due to their ability to model context and benefit from transfer learning (Vaswani et al., 2017; Devlin et al., 2019). Following this paradigm, we fine tune and compare three popular pretrained Transformer models using the same preprocessing and training pipeline in order to ensure a fair comparison.

In addition to classification performance, efficiency is an important consideration for practical applications. Large Transformer models often require significant computational resources, which can limit their usability in real world settings. For this reason, after selecting the best performing model, we apply model compression techniques and evaluate the trade off between accuracy and efficiency. Specifically, we explore knowledge distillation and quantization, and compare the compressed models to the original one in terms of predictive performance, inference time, and model size.

Contributions. The main contributions of this work are:

- An end to end pipeline for emotion classification in short texts, including a consistent tokenizer based preprocessing procedure shared across all models.
- A systematic comparison of three pretrained Transformer architectures under the same training and evaluation setup, with an emphasis on metrics suitable for imbalanced data.
- An empirical analysis of two model compression techniques applied to the best performing model, highlighting the trade off between accuracy, runtime, and model size.

2 Dataset and Preprocessing

The dataset used in this project consists of short textual samples annotated with one of six emotion classes: sadness, joy, love, anger, fear, and surprise. The texts are short and informal, resembling social media posts, and therefore often contain limited context and noisy language. In addition, the dataset is imbalanced, with some emotion classes appearing significantly less frequently than others. The dataset contains **16,000** training samples and **2,000** validation samples, following the predefined split provided with the data, and the original label distribution is preserved for evaluation.

Compared to Part A of the project, where classical neural models were applied, the preprocessing strategy in this work is intentionally different. Pretrained Transformer models are trained on large-scale raw text and rely on subword

Table 1: Class distribution and corresponding class weights in the training set.

Class ID	Emotion	#Samples	Class Weight
0	Sadness	4664	3.42
1	Joy	5350	2.98
2	Love	1299	12.29
3	Anger	2155	7.41
4	Fear	1933	8.26
5	Surprise	568	28.11

tokenization, allowing them to naturally handle casing, punctuation, informal language, and spelling variations. As a result, aggressive preprocessing may remove stylistic and linguistic cues that are important for emotion classification, such as negations or informal expressions.

Based on this motivation, we adopt a minimal preprocessing approach that focuses only on technical data cleaning. Duplicate samples are removed, samples with missing text or labels are discarded, and whitespace normalization is applied. No stemming, lemmatization, stop-word removal, casing normalization, or punctuation normalization is performed.

An explicit sanity check is conducted by evaluating the compatibility of the processed dataset with the tokenizer. Tokenizer sanity checks are performed using the BERT tokenizer with a maximum sequence length of 512 tokens to verify that all texts can be tokenized successfully and that no unknown tokens are produced. Sequence length analysis shows that more than 95% of the samples contain fewer than 45 tokens, and more than 99% contain fewer than 57 tokens, with a maximum length of 87 tokens. During training, a maximum sequence length of 128 tokens is used, under which no samples require truncation.

3 Models and Training Setup

3.1 Models

We evaluate three pretrained Transformer based models for the emotion classification task: BERT (Devlin et al., 2019), ELECTRA (Clark et al., 2020), and DeBERTa (He et al., 2021). All models are initialized from publicly available pretrained checkpoints and adapted to the task using supervised training on the emotion classification dataset.

For each architecture, we use the standard sequence classification head provided by the HuggingFace library. This head consists of a linear layer that maps the final representation of the classification token to six output classes, corresponding to the target emotions. The same classification head structure is used across all models in order to ensure a fair comparison.

The evaluated models differ in their pretraining objectives and architectural design. BERT is pretrained using masked language modeling. ELECTRA employs a discriminative pretraining objective based on replaced token detection. DeBERTa introduces disentangled attention and enhanced positional representations. These differences allow us to analyze how various design choices influence emotion classification performance on short text inputs.

3.2 Training

We trained all models using a supervised fine-tuning setup for the six-class emotion classification task. The training was conducted on the predefined training split, and we based all model selection decisions strictly on the validation set performance.

To handle the clear class imbalance in the dataset, we used a weighted cross-entropy loss during training. These weights were calculated based on the inverse frequency of each emotion in the training data and remained fixed throughout the process. For validation, we used a standard unweighted loss to evaluate the models under the natural distribution of the data.

For our baseline comparison, we used the AdamW optimizer without a learning rate scheduler to ensure a clear and direct comparison across the different architectures. We also applied gradient clipping with a maximum norm of 1.0 to

Table 2: Optimal training configurations selected for each model following the grid search procedure.

Model	Learning Rate	Batch Size	Weight Decay	Epochs	Encoder Fine-Tuning
BERT	2e-5	16	0.00	4	Full
ELECTRA	3e-5	32	0.00	4	Full
DeBERTa	2e-5	32	0.01	3	Full

maintain training stability. However, after identifying the best-performing model, we conducted further experiments using a linear learning rate scheduler with a 10% warmup phase to see if we could further optimize the results.

We selected our hyperparameters through a focused grid search over several key settings: learning rates (2e-5, 3e-5), batch sizes (16, 32), weight decay (0.0, 0.01), and training duration (3, 4 epochs). We intentionally kept these ranges modest to avoid overfitting the high-capacity Transformer models on our relatively small dataset.

Additionally, we evaluated two different training strategies for each model: full fine-tuning of the entire encoder versus partial fine-tuning, where the encoder parameters were frozen and only the classification head was trained. This allowed us to measure the actual performance gain compared to the computational cost of training the full model.

Throughout the process, we saved the checkpoint that achieved the best validation performance for each configuration. We also tracked practical metrics like total training time, inference latency, and peak GPU memory usage to compare the efficiency of the models. All experiments were run using a fixed random seed of 42 to ensure the results can be fully reproduced.

4 Evaluation Metrics

We evaluate our models using a mix of predictive and efficiency metrics. This allows us to measure not just how accurate the models are, but also their practical cost for real-world use. Since our dataset is clearly imbalanced, we chose metrics that give us a fair picture of how the models perform on both frequent and rare emotions.

- **Accuracy**: We report the overall proportion of correct predictions. While Accuracy is easy to understand, we treat it with caution because a model could achieve high accuracy just by guessing the majority class most of the time.
- **Macro-F1 Score**: This is our most important metric for performance. Macro-F1 calculates the F1-score for each emotion separately and then averages them. By giving equal weight to every class, it ensures that a good score actually requires the model to identify 'Surprise' or 'Love' just as well as it identifies 'Joy' (Davis and Goadrich, 2006).
- **Weighted-F1 Score**: This acts as a middle ground, averaging the F1-scores while weighting them by how often each class appears in the validation set.
- **Confusion Matrix**: We use this to get a detailed look at where the model gets confused. It helps us see, for example, if the model is accidentally labeling 'Love' as 'Joy' or if it's struggling with the smaller classes.
- **Efficiency Metrics**:
  - **Training Time**: Reported as the average time per epoch.
  - **Peak GPU Memory**: The maximum memory used during training, which tells us how "heavy" the model is.
  - **Inference Latency**: The average time it takes to process a single sample (in milliseconds). This is critical for knowing if the model can work in real-time.

5 Results

5.1 Model Comparison

Table 3 presents a quantitative comparison between the three pretrained Transformer models using their optimal training configurations, as selected through the grid search procedure. All predictive performance metrics are reported on the validation set, while efficiency-related measurements reflect the computational cost of training and inference.

Table 3: Comparison of the optimal configurations for each Transformer model on the validation set.

Model	Accuracy	Macro-F1	Weighted-F1	Train Time per epoch (s)	Latency (ms)	GPU Memory (MB)
BERT	0.9399	0.9204	0.9411	47.95	1.04	2579
ELECTRA	0.9369	0.9152	0.9378	37.94	1.09	4488
DeBERTa	**0.9414**	**0.9196**	**0.9426**	48.05	1.31	7500

All three models achieve strong performance on the emotion classification task, exceeding the minimum accuracy requirements defined in the project guidelines.It is worth noting that the dataset is relatively clean and well-labeled, which partially explains the high absolute accuracy values observed across all models. Among the evaluated architectures, DeBERTa demonstrates the strongest overall performance. While BERT attains a marginally higher Macro-F1 score, the difference between the two models is negligible. DeBERTa achieves the highest validation accuracy and Weighted-F1 score, indicating superior performance under the natural class distribution.

ELECTRA exhibits competitive results while offering faster training and inference times. However, its lower Macro-F1 score suggests weaker performance on minority emotion classes compared to BERT and DeBERTa.

5.2 Analysis of Results

Looking at the results, we can see a clear trade-off between predictive power and hardware efficiency. In our tests, DeBERTa's unique attention mechanism and enhanced positional encoding gave it a real advantage across most metrics, though it definitely required more GPU memory to run.

We noticed that BERT actually achieved a slightly higher Macro-F1 score, but the gap is less than 0.1%, which is basically negligible in a practical setting. When looking at the total trade-off, we believe DeBERTa is the superior model because it consistently outperformed the others in both overall Accuracy and Weighted-F1.

On the other hand, while ELECTRA was much more efficient and faster to train, its lower Macro-F1 shows that it struggled more with the minority emotion classes that are harder to identify. Because of its high accuracy and more modern architecture, we chose DeBERTa as our reference model for the subsequent compression experiments.

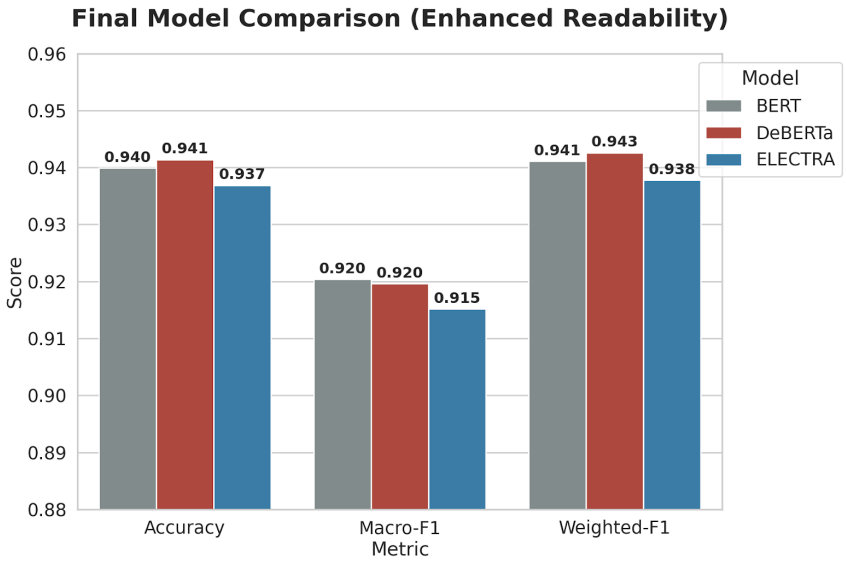

Figure 1: Comparison of validation performance across the three Transformer models using their optimal configurations.

6 Best Model Compression

After comparing the three Transformer models, we chose DeBERTa-base as our reference model for the compression phase. Even though its performance lead was small, DeBERTa was the most "expensive" model in our tests, especially regarding GPU memory and inference logic. This made it the perfect candidate for compression; we wanted to see how much of its resource footprint we could cut down while trying to keep its high predictive accuracy intact.

To do this, we experimented with two different but complementary techniques: knowledge distillation and quantization. These represent two very different ways to make a model more efficient. In knowledge distillation, we focus on "teaching" a smaller model to act like our fine-tuned DeBERTa model during training. Quantization, on the other hand, is a more direct approach where we reduce the numerical precision of the model's parameters to save space and speed up inference. In the following sections, we evaluate both methods and compare them directly to our original FP32 DeBERTa baseline.

6.1 Knowledge Distillation

Knowledge distillation is a training-based compression method in which a smaller student model is trained to mimic the behavior of a larger teacher model (Hinton et al., 2015). In our setup, the fine-tuned DeBERTa-base model serves as the teacher, while a compact Transformer model is used as the student.

The student is trained using a combined objective consisting of two loss terms: a standard cross-entropy loss with respect to the ground truth labels, and a Kullback–Leibler divergence loss between the softened output distributions of the teacher and student models. The final loss is defined as:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{CE} + (1 - \alpha) \cdot \mathcal{L}_{KL},$$

where α controls the trade-off between label supervision and teacher guidance. The weighting coefficient is set to $\alpha = 0.3$, placing greater emphasis on the distillation loss in order to encourage closer alignment with the teacher's output distribution. The temperature parameter used to soften the output distributions is set to $T = 2$, following common practice in the literature (Hinton et al., 2015).

Class-weighted cross-entropy is used in the supervised loss term in order to handle the class imbalance in the dataset. The student model is trained for five epochs using the AdamW optimizer, with a learning rate of 2×10^{-5} and a batch size of 16.

The student architecture is chosen as DeBERTa-v3-small, which belongs to the same model family as the teacher while employing a substantially reduced hidden dimension. This choice enables isolating the effect of architectural compression without introducing differences in attention mechanisms or embedding design. By remaining within the DeBERTa family, the distillation process focuses on transferring representational knowledge rather than compensating for architectural mismatches.

During training, the student model checkpoint achieving the best Macro-F1 score on the validation set is retained. This ensures that the distilled model is optimized for balanced performance across emotion classes.

6.2 Quantization

Quantization is an inference-time compression technique that reduces the numerical precision of model parameters, thereby decreasing memory usage (Jacob et al., 2018). In contrast to knowledge distillation, quantization does not require retraining the model.

We apply post-training dynamic quantization to the fine-tuned DeBERTa-base model. Specifically, the linear layers of the model are quantized from 32-bit floating-point precision to 8-bit integer precision. Dynamic quantization is chosen due to its simplicity and robustness, as it does not require calibration data and can be applied directly to a trained model.

Quantization is performed on the CPU backend, which is the standard deployment setting for dynamic quantization in PyTorch. The resulting quantized model is evaluated using the same validation protocol as the original model, allowing for a direct comparison of predictive performance and memory efficiency.

Model	Accuracy	Macro-F1	Weighted-F1	#Params (M)	Size (MB)
DeBERTa-base (FP32)	**0.9414**	**0.9196**	**0.9426**	139	531
DeBERTa-v3-small (KD)	0.891	0.875	0.892	71	270
DeBERTa-base (INT8)	0.9064	0.8833	0.9079	139	246

Table 4: Comparison between the original DeBERTa-base model and two compression techniques: knowledge distillation (KD) and post-training INT8 quantization. Model size (MB) and parameter counts are measured from the trained checkpoints used in our experiments (serialized weights and model parameter tensors). Inference latency and peak GPU memory are omitted to ensure an apples-to-apples comparison, as PyTorch post-training dynamic INT8 quantization is supported only on CPU backends.

6.3 Comparison of Compressed Models

Table 4 summarizes the performance and efficiency of the original DeBERTa model and its compressed variants. In addition to predictive metrics, we report model size and parameter count in order to quantify the efficiency gains achieved through compression.

The distilled model achieves competitive performance while significantly reducing model complexity. Although there is a noticeable drop in Macro-F1 compared to the original model, the student retains strong overall accuracy and balanced performance across classes. This demonstrates that a substantial portion of the teacher's knowledge can be transferred to a smaller model.

Quantization introduces minimal changes to the model architecture and training procedure, making it attractive for deployment scenarios. While quantization leads to a moderate decrease in predictive performance, its primary benefit lies in reducing the memory footprint. The results highlight that quantization provides a favorable trade-off when strict memory constraints are present.

6.4 Discussion

The compression experiments reveal distinct trade-offs between the two approaches. Knowledge distillation provides a training-time solution that yields a substantially smaller student model with competitive predictive performance, at the cost of additional training complexity. In contrast, post-training quantization offers a lightweight and deployment-oriented optimization that reduces the model's memory footprint without requiring retraining.

Overall, both techniques successfully reduce the computational demands of the original DeBERTa model while maintaining strong emotion classification performance. These results demonstrate that Transformer-based models can be effectively adapted to resource-constrained settings through principled compression strategies.

7 Conclusion

In this project, we successfully developed an emotion classification system for short texts that exceeded 94% accuracy while maintaining strong balanced performance across emotion classes. By fine-tuning three different Transformer architectures on a dataset of 16,000 samples, we found that DeBERTa provided the most robust performance, particularly when handling class imbalance through weighted cross-entropy. Our experiments showed that full fine-tuning is essential for capturing the nuances of emotional language in tweets.

Additionally, our work on model compression proved that these high-capacity models can be made much more efficient. Through INT8 quantization, we reduced the model size by over 50% while maintaining most of its predictive power. Ultimately, this project demonstrates that while modern Transformers are resource-intensive, they can be effectively adapted and compressed for practical, real-world NLP applications.

A Appendix

A.1 Confusion Matrix of the Best Performing Model

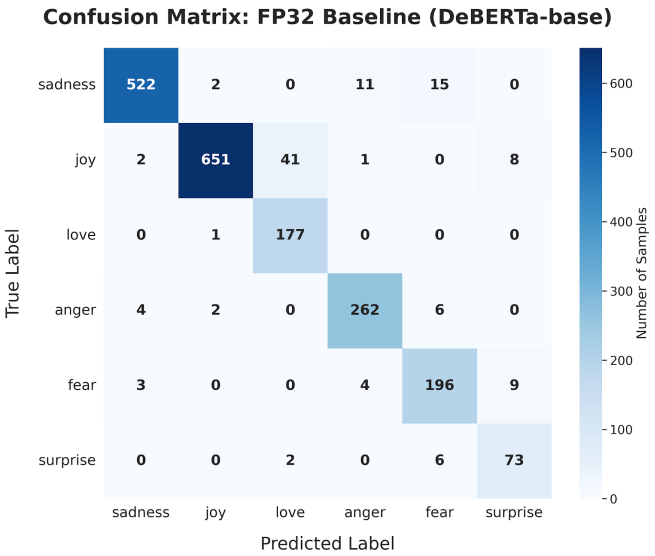

Figure 2: Confusion matrix of the FP32 DeBERTa-base model on the validation set. Rows correspond to the true emotion labels, while columns correspond to the predicted labels. The strong diagonal indicates high classification accuracy across most emotion classes. Most misclassifications occur between semantically related emotions such as *joy* and *love*, and between *fear* and *surprise*, reflecting the inherent ambiguity of short emotional texts.

B Training Dynamics of the Best Model Configurations

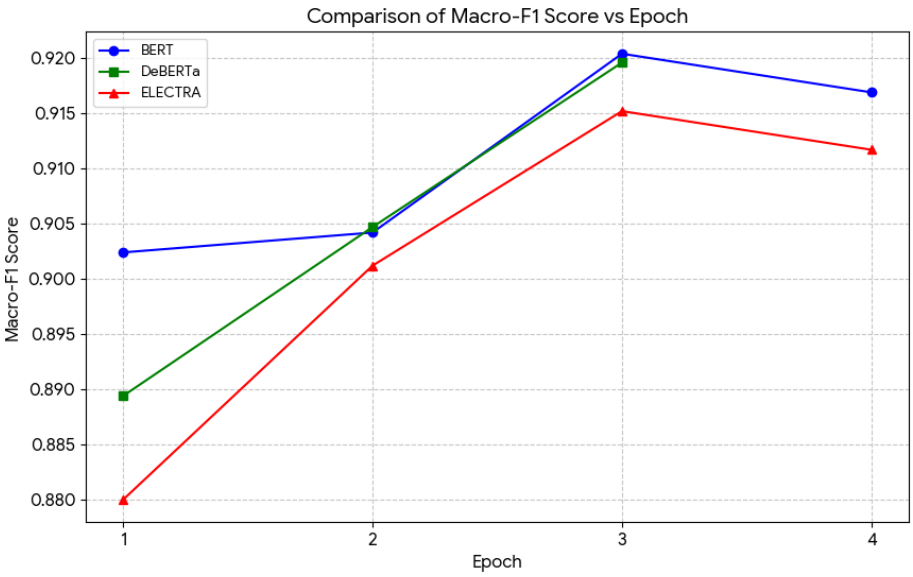

Figure 3: Comparison of Macro-F1 score across training epochs for the best-performing configuration of each Transformer model. Each curve represents the optimal hyperparameter setting selected for BERT, DeBERTa, and ELECTRA. While BERT achieves the highest Macro-F1 score overall, DeBERTa exhibits nearly identical performance, with differences well below 0.1%. This close performance indicates that both models handle class imbalance effectively, while ELECTRA consistently trails behind, particularly in earlier epochs. Based on this comparison and DeBERTa's superior weighted performance under the natural label distribution, DeBERTa was selected as the reference model for subsequent compression experiments.

References

Clark, K., Luong, M.-T., Le, Q. V., and Manning, C. D. (2020). Electra: Pre-training text encoders as discriminators rather than generators. In *Proceedings of ICLR*.

Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and roc curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*.

He, P., Liu, X., Gao, J., and Chen, W. (2021). Deberta: Decoding-enhanced bert with disentangled attention. In *Proceedings of ICLR*.

Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., and Kalenichenko, D. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Vaswani, A. et al. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*.